

SYSTEM SOFTWARE — VIVA MASTER GUIDE

30 Lab Questions • Code flow • OS/Linux concepts • Follow-up viva questions

How to use this tonight: For every random question, be able to explain (1) what the program is trying to do, (2) the execution flow, (3) what each important system/library call does, (4) what happens inside the OS at a high level, and (5) the likely “what if?” questions.

Source basis: The uploaded lab sheet lists 30 exercises covering file/process management and process management.

Q1 — 1. Links and FIFO

Core: A soft link is a separate filesystem object that refers to a pathname. A hard link is another directory entry for the *same inode*. A FIFO is a named pipe used for IPC.

Commands: `ln -s` original soft • `ln` original hard • `mkfifo myfifo`.

Calls: `symlink(target, linkname)` • `link(old,new)` • `mkfifo(path,mode)`. `mknod` can also create special files at a lower level.

Flow: syscall/library interface → kernel → filesystem object/directory entry → return success or -1 with errno. 0666 is filtered by umask.

Difficulty	Viva target
Medium	Explain flow + OS concept + follow-ups

Question: 1. Links and FIFO — Create soft link, hard link and FIFO using shell commands and system calls.

Q: Why can a symlink point to a missing file?

A: It stores a pathname/reference; target resolution happens later. Such a link is dangling.

Q: Why can a hard link survive deletion of the original name?

A: Both names point to the same inode. Data remains while at least one directory entry/reference keeps the inode alive.

Q: What is a FIFO?

A: A named pipe: a kernel-managed IPC endpoint represented in the filesystem; readers and writers communicate through it.

Q: What is the big difference?

A: Soft link → path/reference and separate inode. Hard link → same inode. FIFO → IPC object, not ordinary persistent file contents.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or “what if?”

Q2 — 2. Background infinite loop and /proc

Flow: `fork` is not required here; run the program with `&`. The program calls `getpid()`, prints its PID, then repeatedly sleeps. `sleep(1)` prevents a busy loop.

/proc: a virtual filesystem populated by the kernel. `/proc/PID/status` shows process state/PID/PPID and other information; `/proc/PID/fd` shows open descriptors; `/proc/PID/exe`, `/proc/PID/cwd`, `/proc/PID/maps`, `/proc/PID/stat` and others expose process information.

Difficulty	Viva target
Easy	Explain flow + OS concept + follow-ups

Question: 2. Background infinite loop and /proc — Run an infinite-loop process in the background and inspect its /proc directory.

Q: Why does /proc/PID disappear after the process exits?

A: It represents live kernel process information; once the process is gone, its proc entry is gone.

Q: Why sleep?

A: To keep the process alive while avoiding unnecessary CPU consumption.

Q: What does State=S usually mean here?

A: Sleeping; the process is waiting in sleep rather than executing.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or “what if?”

Q3 — 3. creat() and file descriptor

Core: `creat(path,mode)` creates a file or opens an existing one for writing and truncates it. It returns a small integer FD, or -1 on error.
FD basics: 0=stdin, 1=stdout, 2=stderr. If all are occupied, the next commonly available FD is 3, but the rule is actually the *lowest available* descriptor.

Difficulty	Viva target
Easy	Explain flow + OS concept + follow-ups

Question: 3. creat() and file descriptor — Create a file and print its file descriptor.

Q: What does creat do if the file exists?

A: It truncates it to zero length and opens it for writing.

Q: What does 0644 mean?

A: Owner read/write; group read; others read, subject to umask.

Q: What is an FD?

A: A per-process integer handle referring to an open kernel object/open file description.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or “what if?”

Q4 — 4. open O_RDWR and O_EXCL

O_RDWR: read + write; does not create a missing file.
O_CREAT: create if absent. **O_EXCL + O_CREAT:** fail if the pathname already exists, typically with EEXIST. The mode argument matters when a new file is created.
Important: access mode (O_RDONLY/O_WRONLY/O_RDWR) is different from filesystem permission bits such as 0644.

Difficulty	Viva target
Medium	Explain flow + OS concept + follow-ups

Question: 4. open O_RDWR and O_EXCL — Open an existing file read/write and try exclusive creation.

Q: What if existing.txt is missing in the first open?

A: O_RDWR alone fails; it does not create.

Q: What does O_EXCL really guarantee?

A: With O_CREAT, it requests exclusive creation so an existing pathname causes failure; it is not a read/write permission flag.

Q: Why can O_EXCL be useful?

A: It prevents two creators from both believing they exclusively created the same pathname.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or “what if?”

Q5 — 5. Five files and /proc/PID/fd

Open five files with `O_CREAT|O_RDWR`. Store the returned descriptors in an array. If 0/1/2 are occupied, they will commonly be 3–7, but descriptors are always allocated from the lowest available numbers, so this is not guaranteed.
`/proc/PID/fd` contains symbolic links showing what each open descriptor refers to. The infinite loop keeps the process and descriptors alive for inspection.

Difficulty	Viva target
Medium	Explain flow + OS concept + follow-ups

Question: 5. Five files and `/proc/PID/fd` — Create five files, keep the process alive, and inspect its FD table.

Q: Why store the FDs in an array?

A: So each descriptor can be retained and associated with its file.

Q: What happens when the process exits?

A: Its descriptors are closed by the kernel and the `/proc/PID` entry disappears.

Q: What does `fd/` show?

A: Per-process descriptor entries, often as links to files, pipes, sockets, `/dev` objects, etc.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or “what if?”

Q6 — 6. read STDIN and write STDOUT

FDs: STDIN_FILENO=0, STDOUT_FILENO=1. `read(fd,buf,n)` returns bytes read: >0 data, 0 EOF, -1 error. `write` can return fewer bytes than requested, so robust code loops until the whole chunk is written.
 Important: read/write are byte-oriented, not line-oriented. A buffer of 1024 means “request up to 1024 bytes per call”, not a universal system limit.

Difficulty	Viva target
Medium	Explain flow + OS concept + follow-ups

Question: 6. read STDIN and write STDOUT — Copy bytes from stdin to stdout using only read/write.

Q: Why ssize_t?

A: It can represent a positive byte count, zero, and -1 for error.

Q: Why might write be partial?

A: The destination may accept only part of the requested data, especially pipes/sockets/other non-regular destinations.

Q: stderr vs stdout?

A: stderr is FD 2 and is typically unbuffered; stdout is FD 1 and is commonly line-buffered on a terminal and fully buffered when redirected.

Q: Why fprintf(stderr,...)?

A: It sends diagnostic/error text to stderr, so stdout can remain clean and can be redirected independently.

Q: What does perror do?

A: It prints a supplied prefix plus a human-readable message based on the current errno.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or “what if?”

Q7 — 7. Copy file1 to file2

Open source `O_RDONLY`. Open destination `O_WRONLY|O_CREAT|O_TRUNC` with a mode such as `0644`. Repeatedly read chunks and write them, handling partial writes. Close both FDs.
`O_TRUNC` clears an existing destination. This simple program copies file contents; full GNU `cp` also handles many metadata and special-file cases.

Difficulty	Viva target
Medium	Explain flow + OS concept + follow-ups

Question: 7. Copy file1 to file2 — Implement cp-like copying using open/read/write.

Q: Why `O_TRUNC`?

A: So an old longer destination is not left with stale bytes after the new shorter contents.

Q: Why loop read/write?

A: Files can be larger than the buffer, and write can be partial.

Q: Why `argc/argv`?

A: The source and destination are supplied as command-line arguments.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or “what if?”

Q8 — 8. fdopen and line-by-line reading

Typical flow: `open` → `FD` → `fdopen` → `FILE*` → `fgets` → `fputs` → `fclose`.
`fdopen` associates an existing `FD` with a `stdio` stream; it does not perform another `open`.
`fgets` reads at most `size-1` characters, retains a newline if one was read, null-terminates the string, and returns `NULL` at `EOF/error`.
`fputs` does not add a newline.
`fclose` closes the stream and underlying `FD`.

Difficulty	Viva target
Medium	Explain flow + OS concept + follow-ups

Question: 8. `fdopen` and line-by-line reading — Open a file read-only, read line by line, print each line, and close at `EOF`.

Q: Why `fdopen` instead of `fopen`?

A: The program already has an `FD` from `open` and wants to use `stdio` functions such as `fgets`.

Q: What if a line is longer than the buffer?

A: `fgets` returns it in pieces; multiple calls may be needed for one logical line.

Q: Should we close the `FD` after `fclose`?

A: No; successful `fclose` closes the underlying descriptor.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or “what if?”

Q9 — 9. stat() metadata

`stat(path,&s)` asks the kernel/filesystem for metadata and fills `struct stat`. It does not need to read the file's contents.
Key fields: `st_ino` inode number; `st_nlink` hard-link count; `st_uid/st_gid` owner/group; `st_size` logical size; `st_blksize` preferred I/O block size; `st_blocks` allocated blocks (Linux commonly in 512-byte units); `st_atime` last access; `st_mtime` content modification; `st_ctime` inode/status change time—not creation time.
`stat` vs `lstat`: `stat` follows the final symlink; `lstat` reports the symlink itself.

Difficulty	Viva target
Medium	Explain flow + OS concept + follow-ups

Question: 9. `stat()` metadata — Print inode, links, UID/GID, size, block size/count and timestamps.

Q: Where does the information come from?

A: The pathname is resolved by the kernel; filesystem metadata associated with the inode/filesystem object is used to fill `struct stat` and copied to user memory.

Q: Does `st_size` equal allocated storage?

A: Not necessarily. Sparse files can have a large logical size but fewer allocated blocks.

Q: Is `st_ctime` creation time?

A: No. On Linux it is the last change to inode/status metadata, not a birth/creation timestamp.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or “what if?”

Q10 — 10. lseek and sparse file

After writing 10 bytes, the offset is 10. `lseek(fd,10,SEEK_CUR)` moves it to 20 and returns the new offset. No data is written by `lseek`. The next 10-byte write starts at offset 20, leaving a logical hole from offsets 10–19.
A sparse file can have logical size 30 while not allocating physical blocks for every hole byte. `od` can show the logical zero-filled hole; tools such as `ls -l` vs `ls -ls` help distinguish logical size and allocated blocks.

Difficulty	Viva target
Medium	Explain flow + OS concept + follow-ups

Question: 10. `lseek` and sparse file — Write 10 bytes, move the file offset forward 10 bytes, then write another 10 bytes.

Q: Why check `lseek` against `(off_t)-1`?

A: That is the failure return value; offsets use `off_t`.

Q: What does `SEEK_SET/CUR/END` mean?

A: Offset from beginning/current position/end.

Q: What is a hole?

A: A range in the logical file with no actual written data; reading it returns zeros, while storage may be unallocated.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or “what if?”

Q11 — 11. dup, dup2 and fcntl duplication

Critical distinction: duplicated FDs are different integers in the process FD table, but they refer to the same open file description. The open file description stores the current offset and file status flags.

`dup(fd)` returns the lowest available FD. `dup2(fd,newfd)` makes newfd refer to the same open file description, closing/replacing newfd first if needed.

`fcntl(fd,F_DUPFD,min)` returns the lowest available FD $\geq$ min. If `O_APPEND` was used, each write is positioned at the end. Two descriptors created by dup share the underlying offset/state.

Difficulty	Viva target
Hard	Explain flow + OS concept + follow-ups

Question: 11. dup, dup2 and fcntl duplication — Duplicate a descriptor and write through both descriptors.

Q: Do dup and dup2 create another open file description?

A: No. They create another FD referring to the same open file description.

Q: What if the file is opened twice separately?

A: Usually there are separate open file descriptions, so their offsets are independent.

Q: What happens if one duplicate is closed?

A: The other duplicate remains usable; the underlying open state persists until all relevant references are gone.

Q: Why does this matter after fork?

A: Parent and child inherited descriptors can also refer to the same open file description and therefore share the offset.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or “what if?”

Q12 — 12. Find opening mode using fcntl

`fcntl(fd, F_GETFL)` retrieves file status flags associated with the open file description. To extract access mode, use `flags & O_ACCMODE` and compare against `O_RDONLY/O_WRONLY/O_RDWR`. Do not switch on the entire flags value because it can contain unrelated status flags such as `O_APPEND`. Access mode is also different from filesystem permissions like `0644`.

Difficulty	Viva target
Medium	Explain flow + OS concept + follow-ups

Question: 12. Find opening mode using fcntl — Use fcntl to determine whether an FD is read-only, write-only, or read/write.

Q: Why `O_ACCMODE`?

A: It masks out unrelated status flags and leaves the access-mode bits.

Q: If this exact program opened `O_RDWR`, what will it print?

A: Read/write; it is querying the mode it itself requested.

Q: What is `F_GETFL`?

A: A fcntl command for retrieving file status flags.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or “what if?”

Q13 — 13. select for 10 seconds

`select(nfds, readfds, writefds, exceptfds, timeout)` waits for I/O readiness. For stdin only, `nfds` is `STDIN_FILENO+1`, i.e. 1, because `select` considers descriptors 0 through `nfds-1`. It is not the number of descriptors.
FD_SET macros prepare `fd_sets`: `FD_ZERO` clears, `FD_SET` adds, `FD_ISSET` tests after `select`. `readfds` means a read can proceed without blocking (EOF also counts as readable). `writefds` means write readiness. `exceptfds` is for exceptional conditions. `timeout` is maximum wait: `NULL`=infinite, `{10,0}`=up to 10s, `{0,0}`=no wait.
Return: ≥ 0 ready descriptors, 0 timeout, -1 error. `select` only reports readiness; read/write performs the actual I/O. It modifies the `fd_sets`, so repeated calls normally rebuild them.

Difficulty	Viva target
Hard	Explain flow + OS concept + follow-ups

Question: 13. select for 10 seconds — Wait for stdin readiness for up to 10 seconds.

Q: Why `nfds = STDIN_FILENO+1`?

A: `select` expects one more than the highest FD being monitored.

Q: Does `select` read the data?

A: No. It says that a read should not block; the program still calls `read()`.

Q: What does `FD_ISSET` do?

A: Checks whether a descriptor remains marked ready after `select` returns.

Q: What does man 2 `select` mean?

A: Manual section 2 is for system calls; it shows the Linux system-call interface, parameters, return values and errors.

Q: Why is `timeout` called a maximum?

A: The call may return early when a descriptor becomes ready or due to interruption/error.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or “what if?”

Q14 — 14. Determine file type

Use `lstat(path,&s)` and inspect `s.st_mode` with macros such as `S_ISREG`, `S_ISDIR`, `S_ISLNK`, `S_ISFIFO`, `S_ISSOCK`, `S_ISCHR`, `S_ISBLK`.
lstat reports the path itself, so a symlink is recognized as a symlink. stat follows the final symlink. File type is determined from filesystem metadata, not from a filename extension.

Difficulty	Viva target
Medium	Explain flow + OS concept + follow-ups

Question: 14. Determine file type — Take a path from the command line and identify any filesystem file type.

Q: Why lstat rather than stat?

A: To identify a symlink itself. stat would follow the final symlink.

Q: What happens for a broken symlink?

A: lstat can still succeed and report a symlink; stat commonly fails because the target cannot be resolved.

Q: Does the program open the file?

A: No; it obtains metadata with lstat.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or “what if?”

Q15 — 15. environ

`extern char **environ;` refers to the process environment array. Conceptually: `environ` → pointer to `char*` strings → "NAME=value" → ... → NULL.
Loop through pointers and print each string. The environment is normally supplied at process startup and inherited by children. `getenv` retrieves one variable; `setenv/unsetenv` modify the calling process's environment.

Difficulty	Viva target
Medium	Explain flow + OS concept + follow-ups

Question: 15. environ — Display all environment variables using `environ`.

Q: Why `char **`?

A: It is a pointer to an array of `char` pointers; each pointer points to a NAME=value string.

Q: What ends the array?

A: A NULL pointer.

Q: Is `environ` a system call?

A: No; it is a process environment interface/variable exposed by the C runtime.

Q: Can a child change its parent's environment?

A: No. A child's environment is its own process state.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or "what if?"

Q16 — 16. Mandatory locking

Critical viva point: POSIX `fcntl` record locking is normally *advisory*. `F_SETLKW` means "set the lock and wait if it conflicts"; it does not by itself make the lock mandatory. Traditional Linux mandatory locking historically required special filesystem/kernel conditions and file mode bits (classically `setgid` on + group execute off, e.g. 02660), and support/configuration is filesystem/version dependent. Therefore do not claim "fcntl automatically makes locking mandatory."
`F_RDLCK`=shared read lock; `F_WRLCK`=exclusive write lock; `F_UNLCK`=unlock.
`l_whence/l_start/l_len` define the byte range; `l_len=0` means to EOF. `F_SETLK` is nonblocking; `F_SETLKW` waits.

Difficulty	Viva target
Hard	Explain flow + OS concept + follow-ups

Question: 16. Mandatory locking — Implement read and write locking with `fcntl`; understand the mandatory/advisory distinction.

Q: Can multiple read locks coexist?

A: Yes, read locks can coexist when the ranges are compatible.

Q: Can a write lock coexist with a read lock?

A: No; conflicting ranges block/conflict.

Q: Is the lock on the filename?

A: No. It is a byte-range/record lock associated with the file/opening and process semantics.

Q: What releases the lock?

A: Explicit unlock and the relevant close/process termination semantics release `fcntl` locks.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or "what if?"

Q17 — 17. Online ticket reservation

Use an exclusive `F_WRLCK` with `F_SETLKW` so only one reservation process updates the number at a time. Seek to the beginning before reading and again before rewriting. Store the ticket as text (e.g. "100\n"), parse it with `atoi`, increment, then write the new value. Truncate the file to the new length so stale old digits are not left behind.
Race without lock: A reads 100; B reads 100; both write 101. With lock: A gets 101, unlocks; B then reads 101 and writes 102.

Difficulty	Viva target
Hard	Explain flow + OS concept + follow-ups

Question: 17. Online ticket reservation — Lock a ticket file, read the current number, increment it, and safely update it.

Q: Why write lock?

A: The operation modifies shared data, so it must exclude other readers/writers in the protected region.

Q: Why lseek before reading/writing?

A: The file offset must be positioned at the desired location; after reading it may be at EOF.

Q: Why ftruncate?

A: If the replacement value is shorter than the old content, it removes leftover bytes.

Q: What does `l_len=0` mean?

A: Lock from `l_start` through the end of the file.

Q: What if the lock is removed?

A: Concurrent processes can read the same old value and generate duplicate ticket numbers.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or "what if?"

Q18 — 18. Record locking

With fixed-size records of 64 bytes: record 0 starts at 0, record 1 at 64, record 2 at 128. Set `l_start = r*SZ` and `l_len=SZ` to lock exactly one record. Writer uses `F_WRLCK`; reader uses `F_RDLCK`. `pwrite/pread` access a specified offset without changing the process's current file offset. Different records can therefore be accessed concurrently; the same record conflicts appropriately.

Difficulty	Viva target
Hard	Explain flow + OS concept + follow-ups

Question: 18. Record locking — Create three records and lock only the record being accessed.

Q: Why is record locking better than whole-file locking here?

A: Independent records can proceed concurrently; only the record being accessed is protected.

Q: Why `pwrite/pread`?

A: They specify the offset directly and do not alter the current file offset.

Q: Can two readers access one record?

A: Yes, shared read locks can coexist.

Q: Can a reader and writer access the same locked record simultaneously?

A: Not when their locks conflict.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or “what if?”

Q19 — 19. Measure getpid with TSC

Read the timestamp counter before and after N repeated calls: `end-start` is total TSC ticks observed; divide by N for an average per call. A large N amortizes measurement overhead/noise.
`__rdtsc()` is a compiler intrinsic, not a normal syscall. The result is cycles/ticks, not automatically nanoseconds. Modern glibc may cache/getpid in user space on some systems, so the benchmark measures the system's getpid interface as implemented, not necessarily a kernel transition on every iteration.

Difficulty	Viva target
Hard	Explain flow + OS concept + follow-ups

Question: 19. Measure getpid with TSC — Estimate average time/cost of getpid using the CPU timestamp counter.

Q: Why one million iterations?

A: To reduce the relative impact of loop/timestamp overhead and scheduling noise.

Q: Why uint64_t?

A: The timestamp counter is a large nonnegative counter.

Q: Is the result directly nanoseconds?

A: No. Converting cycles/ticks to time requires frequency information and careful benchmarking.

Q: What adds measurement error?

A: Timestamp read overhead, loop overhead, scheduling/preemption, CPU migration/frequency behavior and lack of serialization.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or “what if?”

Q20 — 20. Nice and process priority

`getpriority(PRIO_PROCESS,0)` queries the calling process's nice value. On Linux, the usual nice range is -20 to +19: lower nice means higher preference under normal scheduling; higher nice means lower preference. The shell `nice -n 10 ./program` starts it with an adjusted niceness. Changing nice is not the same as switching to SCHED_FIFO/RR. Ordinary users can generally make their own process less favored (increase nice); increasing priority by decreasing nice can require privilege.

Difficulty	Viva target
Medium	Explain flow + OS concept + follow-ups

Question: 20. Nice and process priority — Find the running program's nice value and modify it with the nice command.

Q: Does nice guarantee CPU percentage?

A: No; scheduling also depends on runnable tasks, policy, CPUs and other factors.

Q: Is nice the same as real-time priority?

A: No. Nice is associated with normal scheduling; FIFO/RR use real-time priority.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or "what if?"

Q21 — 21. fork parent/child IDs

`fork()` creates a child. Return value: -1 failure; 0 in child; child's PID in parent. Both continue from the instruction after fork.
Child can print `getpid()` and `getppid()`; parent can print its `getpid()` and the returned child PID. Output order is not guaranteed because the scheduler decides which process runs first. Modern fork uses copy-on-write for memory pages.

Difficulty	Viva target
Easy	Explain flow + OS concept + follow-ups

Question: 21. fork parent/child IDs — Call fork and print parent and child process IDs.

Q: Why different return values?

A: So parent and child can determine which execution path they are in.

Q: Can child output appear first?

A: Yes. Scheduling is not guaranteed.

Q: What does `getppid` return?

A: The current parent PID.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or “what if?”

Q22 — 22. Open before fork, then both write

Because the file is opened *before* fork, the inherited FDs in parent and child refer to the same open file description. That means they share the current file offset and status flags. If child writes first, parent writes after its data; if parent writes first, order reverses. `wait()` after the parent's write does not force child-first; it only makes the parent wait after it has already written. If each process opened the file separately after fork, their open file descriptions would generally be separate.

Difficulty	Viva target
Hard	Explain flow + OS concept + follow-ups

Question: 22. Open before fork, then both write — Open a file, fork, and have parent and child write to it.

Q: Why is FD number the same in both processes?

A: Each process has its own FD table, so the same integer can refer to the inherited shared open file description.

Q: Why is output order unpredictable?

A: The scheduler decides which process executes its write first.

Q: What does close do?

A: It closes that process's descriptor; the shared open state remains while other references exist.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or "what if?"

Q23 — 23. Zombie process

A zombie is a terminated child whose parent has not yet collected its termination status. The kernel retains small bookkeeping such as PID and exit status. The child is not executing.
The demo parent sleeps, allowing to show state Z/defunct. A later wait/waitpid reaps it. This is different from an orphan: an orphan is still running after its parent dies.

Difficulty	Viva target
Medium	Explain flow + OS concept + follow-ups

Question: 23. Zombie process — Create a child that exits while the parent remains alive without waiting.

Q: How is zombie different from orphan?

A: Zombie: child died first, parent alive and hasn't reaped it. Orphan: parent died first, child still runs.

Q: Why `_exit`?

A: It terminates immediately without `stdio/atexit` cleanup; useful after fork in a child when appropriate.

Q: How is zombie removed?

A: The parent calls `wait/waitpid`, or reparenting allows a suitable system process to reap it.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or "what if?"

Q24 — 24. Orphan process

Parent calls fork and exits immediately. Child sleeps so the parent has time to terminate. The child then prints getpid and getppid. Because its original parent is gone, it is reparented/adopted by PID 1 or an appropriate configured subreaper.

Difficulty	Viva target
Easy	Explain flow + OS concept + follow-ups

Question: 24. Orphan process — Create a child that remains alive after the parent exits.

Q: Does the child die when parent exits?

A: No; it continues running.

Q: Is an orphan a zombie?

A: No. The orphan is still executing; a zombie has already terminated.

Q: Why sleep?

A: To ensure the parent exits before the child prints its new PPID.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or “what if?”

Q25 — 25. Three children and waitpid

Store each child PID in `c[0..2]`. Each child prints its PID, sleeps for a different time, then exits with a different status. Parent calls `waitpid(c[1], &st, 0)`, which blocks specifically for child 2. `WIFEXITED` checks normal termination; `WEXITSTATUS` extracts its exit code. Parent then waits for the other children to prevent leftover zombies.

Difficulty	Viva target
Medium	Explain flow + OS concept + follow-ups

Question: 25. Three children and waitpid — Create three children and wait specifically for child 2.

Q: What if child 1 finishes first?

A: Parent remains blocked waiting specifically for child 2. Child 1 can temporarily be a zombie until reaped later.

Q: wait vs waitpid?

A: wait generally waits for any child; waitpid can select a specific PID (or other selection modes).

Q: Why store PIDs?

A: The parent needs the exact PID to wait for a particular child.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or “what if?”

Q26 — 26. Execute an executable with exec

`exec` replaces the current process image; it does not create a new process and normally keeps the same PID. Example: `execl("/bin/echo","echo",name,NULL)`. If `exec` succeeds, it never returns, so code after it executes only on failure. `perror` is therefore placed after `exec`. `argv[0]` conventionally names the program; subsequent entries are its arguments.

Difficulty	Viva target
Medium	Explain flow + OS concept + follow-ups

Question: 26. Execute an executable with exec — Execute another executable and pass command-line input.

Q: fork vs exec?

A: fork creates a new process; exec replaces the current program.

Q: Why NULL?

A: It terminates the variadic argument list for l-style exec calls.

Q: What if execl fails?

A: It returns -1, `errno` is set, and `perror` can report the reason.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or “what if?”

Q27 — 27. execl/execlp/execle/execv/execvp

All five execute the same program; they differ in argument representation, PATH search and environment.
l=list arguments; v=vector/array; p=search PATH; e=explicit environment.
execl: explicit path + list. execlp: PATH + list. execle: explicit path + list + supplied environment. execv: explicit path + array. execvp: PATH + array.

Difficulty	Viva target
Hard	Explain flow + OS concept + follow-ups

Question: 27. execl/execlp/execle/execv/execvp — Execute ls -Rl using all five exec variants.

Q: What does -R mean?

A: Recursive listing.

Q: What does -l mean?

A: Long listing format.

Q: Why use execvp("ls",a)?

A: It supplies arguments as an array and lets the system search PATH for ls.

Q: Does any of these create a process?

A: No; exec replaces the current process image.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or "what if?"

Q28 — 28. Get min/max real-time priority

`sched_get_priority_min(policy)` and `sched_get_priority_max(policy)` query the valid priority range for a scheduling policy. `SCHED_FIFO` and `SCHED_RR` are real-time policies. Exact numeric ranges should be obtained from the system rather than assumed.
This program only queries; it does not change the process policy or priority.

Difficulty	Viva target
Medium	Explain flow + OS concept + follow-ups

Question: 28. Get min/max real-time priority — Get minimum and maximum real-time priorities for `SCHED_FIFO` and `SCHED_RR`.

Q: FIFO vs RR?

A: FIFO does not time-slice equal-priority runnable threads merely because a quantum expires; RR rotates equal-priority threads using a time quantum.

Q: Does this change priority?

A: No, it only reads the supported range.

Q: Is real-time priority the same as nice?

A: No; they belong to different scheduling mechanisms.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or “what if?”

Q29 — 29. Get and modify scheduling policy

`sched_getscheduler(0)` gets the calling process's current policy. `sched_setscheduler(0,target,&sp)` changes the policy and scheduling parameters. For FIFO/RR, `struct sched_param` contains a valid real-time priority; this example uses the minimum for the target policy.
Changing to a real-time policy may require appropriate privileges/capabilities; failure is commonly EPERM for an unprivileged attempt.

Difficulty	Viva target
Hard	Explain flow + OS concept + follow-ups

Question: 29. Get and modify scheduling policy — Read the current policy and change it to SCHED_FIFO or SCHED_RR.

Q: What does 0 mean?

A: The calling process.

Q: What is SCHED_OTHER?

A: The normal/default scheduling policy on typical Linux systems.

Q: What is the difference from Q28?

A: Q28 queries valid priority bounds; Q29 queries the current policy and attempts to change the policy.

Q: Does `sched_getscheduler` change anything?

A: No; it only queries.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or “what if?”

Q30 — 30. Run a script at a specific time with a daemon

Flow: validate HH:MM → fork → parent exits → child calls `setsid` → `chdir("/")` → `umask(0)` → redirect 0/1/2 to `/dev/null` → repeatedly call `time/localtime_r` → when hour/minute match, fork → child `execl("/bin/sh", "sh", script, NULL)` → daemon `waitpid` → sleep to avoid duplicate execution.
setsid() creates a new session and detaches from the old controlling terminal/session. `chdir("/")` avoids holding an arbitrary working directory. `/dev/null` prevents terminal dependency. The second fork is the script child; `exec` replaces it with the shell script.

Difficulty	Viva target
Hard	Explain flow + OS concept + follow-ups

Question: 30. Run a script at a specific time with a daemon — Daemonize a process, check local time, and execute a shell script at a specified HH:MM.

Q: Why `waitpid` after launching the script?

A: To reap the script child and avoid leaving a zombie.

Q: Why sleep 61 after a match?

A: To prevent repeated executions during the same matching minute.

Q: Is this a production scheduler?

A: No; it demonstrates daemonization/scheduling concepts. Production daemons usually add logging, signal handling, robust lifecycle/error handling, privilege management, etc.

Q: Why use cron in real life?

A: cron is an existing scheduling service designed for this job; the lab is demonstrating how a simple scheduler/daemon can be implemented.

Viva answer structure: Start with the purpose → show the execution flow → explain the important call/parameters → explain what the kernel/OS is doing → mention one edge case or “what if?”

LAST-MINUTE LINUX + OS BASICS

What is a process?	A running program plus its execution state/resources: PID, address space, registers/context, open FDs, scheduling information, credentials, etc.
Program vs process	A program is passive code/data on storage. A process is a running instance of that program.
Kernel	The privileged core of the OS that manages CPU, memory, filesystems, devices, processes and provides system-call interfaces.
System call	A controlled interface through which user programs request kernel services, e.g. open, read, write, fork, exec, waitpid.
Library function vs system call	A library function is a user-space API; it may call a system call, wrap it, or do work entirely in user space. Example: printf is stdio; write is a system-call interface.
User mode vs kernel mode	User programs normally run with restricted privileges; kernel mode has the privileges needed to manage protected OS resources.
File descriptor	A small integer in a process's FD table referring to an open resource. 0/1/2 are stdin/stdout/stderr.
Inode	Filesystem metadata object identifying a file and containing metadata; directory entries map names to inode objects.
Directory entry	The filesystem's mapping from a filename to an inode/object; a filename is not the file's data itself.
Open file description	Kernel-side state created by open, including current file offset and status flags; duplicated FDs can share it.
Buffering	stdio may buffer output/input. stdout is commonly line-buffered on a terminal and fully buffered when redirected; stderr is typically unbuffered.
errno/perror	A failed system/library interface commonly returns an error indicator and sets errno. perror prints a prefix plus a readable errno message.
fork/exec/wait	fork creates a child; exec replaces a process image; wait/waitpid collects child termination status and prevents zombies.
Zombie vs orphan	Zombie = terminated child not reaped. Orphan = still-running child whose parent terminated.
PPID/PID	PID identifies a process; PPID identifies its current parent.
Pipe vs FIFO	An anonymous pipe is commonly used between related processes; a FIFO is a named pipe represented in the filesystem and can be opened by unrelated processes.
Hard link vs symlink	Hard link shares inode; symlink stores a pathname/reference.
Blocking	A blocking call may wait until an operation can proceed. select can wait for readiness; read may block until data/EOF depending on object.
Readiness vs data	select says an operation can proceed without blocking; it does not itself consume data.
Permissions vs access mode	open's O_RDONLY/O_WRONLY/O_RDWR controls how that open is used; filesystem mode bits such as 0644 control permission checks.
Race condition	Result depends on timing/interleaving of concurrent operations. Locking protects critical sections.
Advisory vs mandatory locking	Advisory locks work when cooperating programs honor them. Traditional mandatory locking requires specific OS/filesystem conditions and is not automatic just because fcntl is used.
Scheduling	The scheduler decides which runnable thread/process gets CPU time. Normal scheduling and real-time FIFO/RR use different mechanisms.
Nice vs real-time priority	Nice adjusts relative preference under normal scheduling; SCHED_FIFO/RR use real-time priorities.
Daemon	A long-running background service designed to operate independently of an interactive terminal/session.

PATH	A colon-separated list of directories searched by PATH-aware exec functions such as <code>execlp/execvp</code> .
stdin/stdout/stderr redirection	Shell redirection changes where standard descriptors point, e.g. <code>></code> for stdout and <code>2></code> for stderr.

Random-question viva drill: Pick any Q1–Q30. Explain it aloud without reading for 60–90 seconds. Then answer: Why this call? What does each important parameter mean? What happens if it fails? What happens if two processes do it simultaneously? What changes if I remove/replace this call? Where is the relevant information stored (user space/kernel/filesystem)?

Important: The difficulty labels are a study-planning aid, not official TA ratings. The lab sheet itself specifies the 30 questions; this guide expands them with the concepts and follow-ups needed to explain them.